

elasticsearch.



Nathalie Ling

Mardi 21 janvier 2014

Base de données avancées (1) – NFE204, **le cnam**

Présentation d'Elasticsearch

ElasticSearch est un moteur de recherche.

- Open source
- Java
- librairie Lucene
- base NoSQL

La première version a été développée par Shay Banon en février 2010.

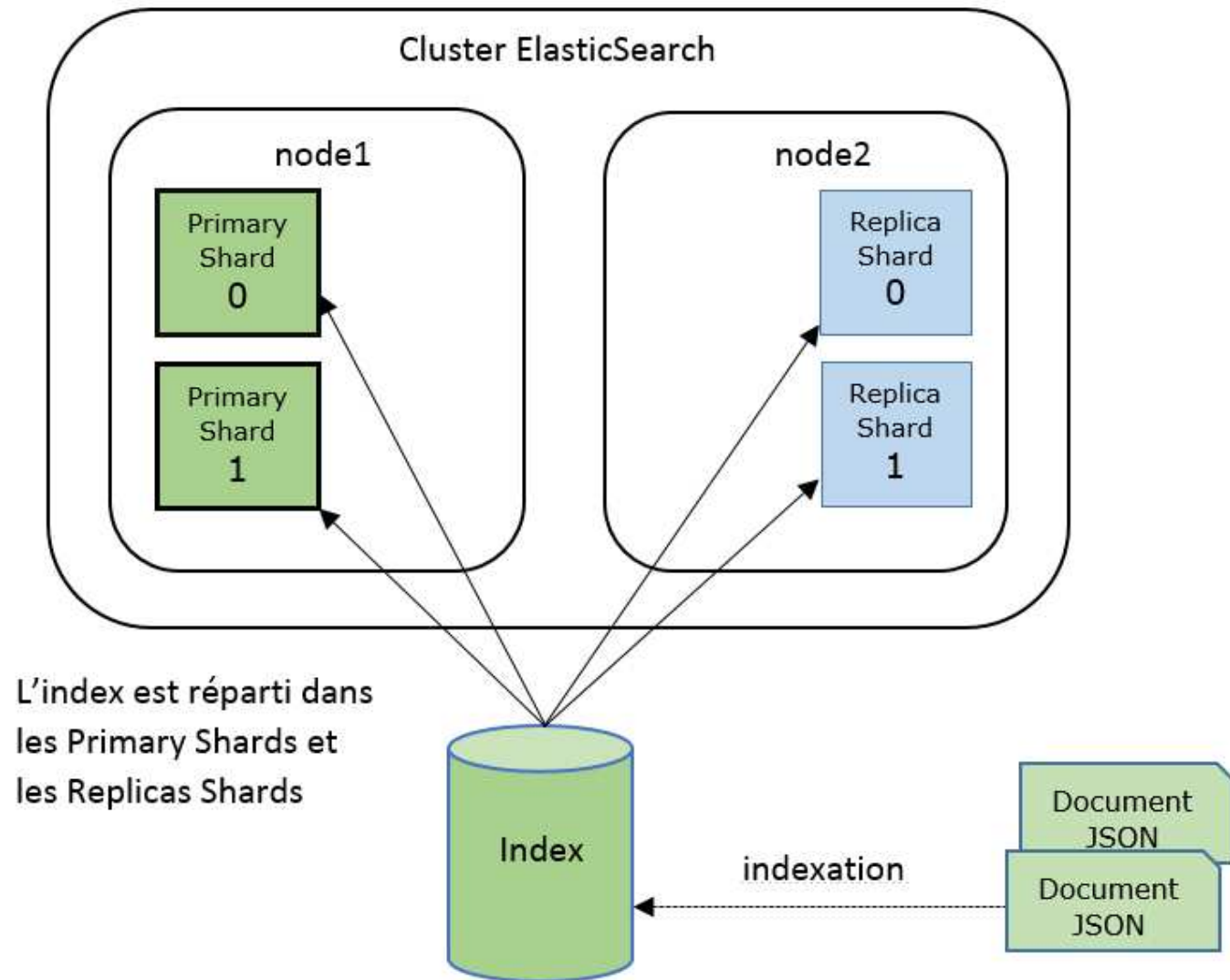
ElasticSearch a deux fonctions :
il sert à insérer des documents JSON dans un index inversé
et à afficher les documents.

- rapide
- fiable
- language Query DSL
- protocole REST
- requêtes HTTP

Il est scalable à l'horizontale.

ElasticSearch est distribué

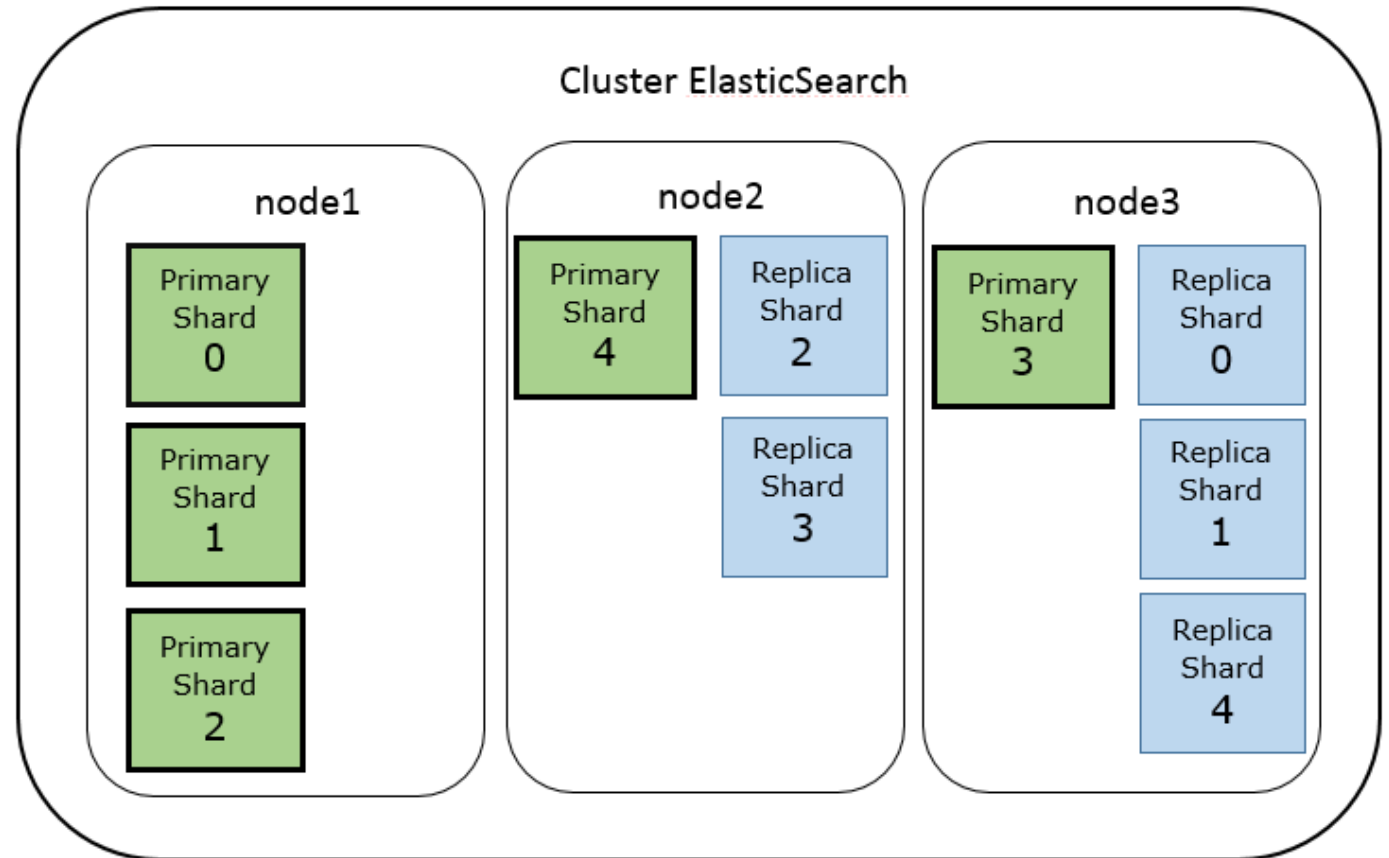
Un index est découpé en fragments (shards).



La haute disponibilité

Par défaut, le nombre de Primary shards est 5 et le nombre de Replica est 1.

On peut avoir un noeud qui tombe en panne et toujours avoir tous les index en ligne.



Un nœud peut avoir 3 rôles :

Le **nœud maître** (Master node) gère l'état du cluster et distribue les shards dans tous les nœuds.

- Le **nœud de données** (Data node). Il contient les shards.
- Le **nœud non Master et non Data**. Toutes les requêtes HTTP qui sont envoyées au cluster passent par ce nœud. Il fait de la répartition de charge (load balancing), il route les requêtes vers les nœuds de données puis rassemble les résultats trouvés dans chaque shard.

Création d'un index

```
curl -XPUT 'http://localhost:9200/biblio'
```

Création d'un document

on indique une URI sous la forme `http://serveur:port/index/type/id`

```
curl -XPUT 'http://localhost:9200/biblio/cd/cd1' -d '{  
    "titre": "Lohengrin",  
    "compositeur": "Richard Wagner"  
}'
```

```
{"ok":true,"_index":"biblio","_type":"cd","_id":"cd1","_version":1}
```

Un document Elasticsearch a un type, un id, une version.

Un type de document est décrit par un schéma.

```
curl -XGET 'http://localhost:9200/biblio/cd/_mapping'
```

```
{ "cd" : {  
  "properties" : { "compositeur" : { "type" : "string" },  
                  "titre" : { "type" : "string" }  
  }  
}
```

On peut avoir plusieurs types de document dans un index.

Affichage du document cd1 de type cd dans ma collection biblio

```
curl -XGET 'http://localhost:9200/biblio/cd/cd1'
```

```
{ "_index": "biblio", "_type": "cd", "_id": "cd1", "_version": 1, "exists": true,
  "_source":
    { "titre": "Lohengrin",
      "compositeur": "Richard Wagner" }
}
```

Recherche d'un document de type cd dans biblio qui a wagner pour compositeur

```
curl -XGET  
'http://localhost:9200/biblio/cd/_search?q=compositeur:wagner'
```

```
... {"_index":"biblio","_type":"cd","_id":"cd1","_score":0.625, "_source" :  
    {  
      "titre": "Lohengrin",  
      "compositeur": "Richard Wagner"  
    }  
}
```

La percolation

On enregistre des requêtes percolateurs dans un index, puis on envoie ces requêtes avec un document, en retour on a les requêtes percolateurs qui correspondent au document.

Fonction : Cela permet de trouver un document qui ne sera pas affiché par une requête percolateur. Puis, de créer une requête non percolateur pour afficher le document ou supprimer le document.

Les requêtes sont dans un index spécifique `_percolator`. Cet index est répliqué sur tous les nœuds.

On crée un index `myindex`

```
curl -XPUT 'http://localhost:9200/myindex'  
{ "ok":true, "acknowledged":true }
```

On crée une requête percolateur

```
curl -XPUT 'localhost:9200/_percolator/myindex/requetewagner' -d '{  
  "query" : {  
    "text" : {  
      "compositeur" : "wagner"}}}'  
  
{ "ok":true, "_index":"_percolator", "_type":"myindex",  
  "_id":"requetewagner", "_version":1 }
```

On crée une autre requête percolateur

```
curl -XPUT 'localhost:9200/_percolator/myindex/requeterossini' -d '{  
  "query" : {  
    "text" : {  
      "compositeur" : "rossini"}}}'
```

```
{"ok":true,"_index":"_percolator","_type":"myindex",  
"_id":"requeterossini","_version":1}
```

On envoie une requête percolateur avec un document

```
curl -XGET 'localhost:9200/myindex/cd/_percolate' -d '{  
  "doc" : {  
    "titre": "Lohengrin",  
    "compositeur": "Richard Wagner"  
  }  
'  
  
{"ok":true,"matches":["requetewagner"]}
```

On envoie la requête avec un autre document

```
curl -XGET 'localhost:9200/myindex/cd/_percolate' -d '{  
  "doc" : {  
    "titre": "Le barbier de Séville",  
    "compositeur": "Gioacchino ROSSINI"  
  }  
}'
```

```
{"ok":true,"matches":["requeterossini"]}
```

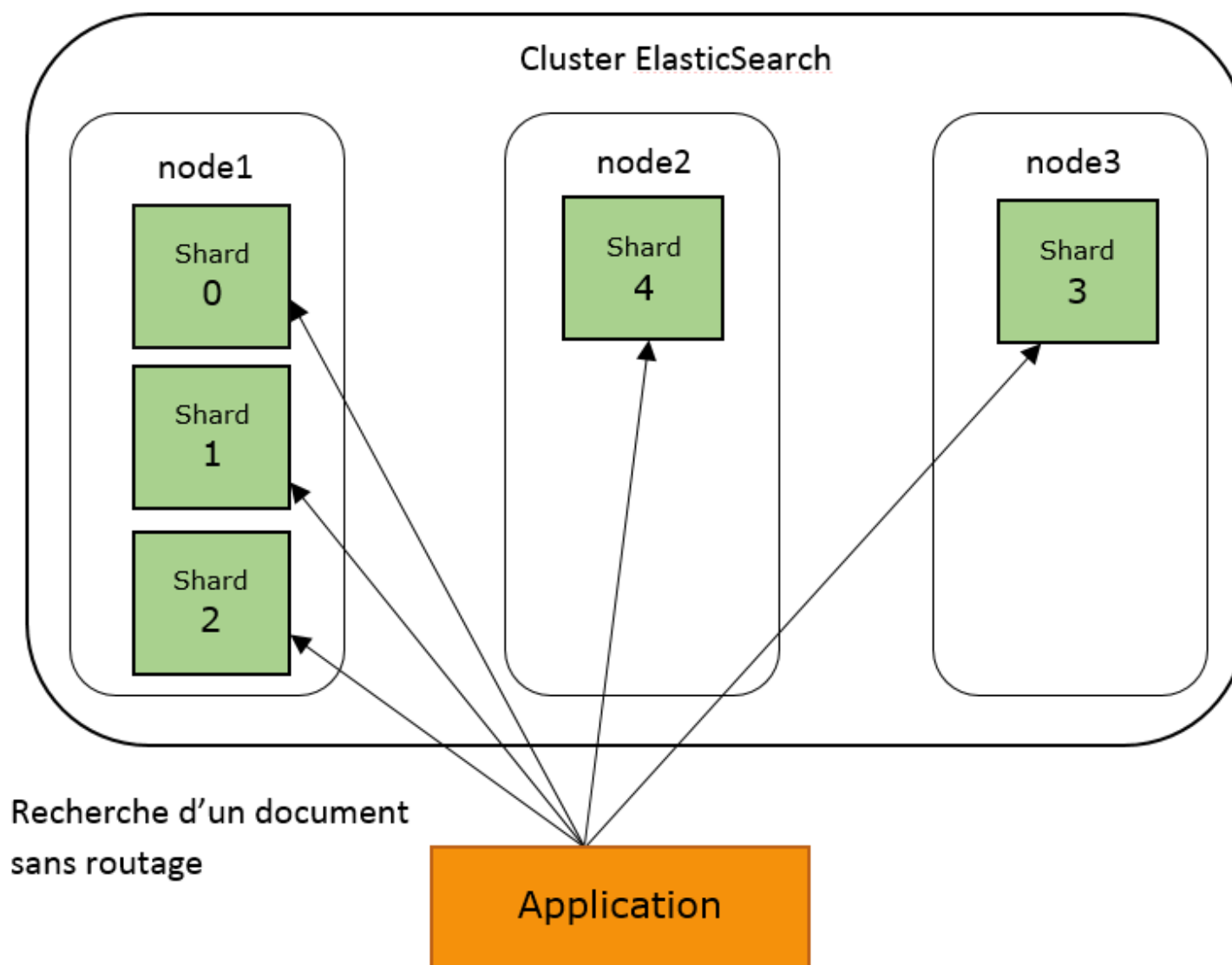
La distribution des documents, utilisation du paramètre routing

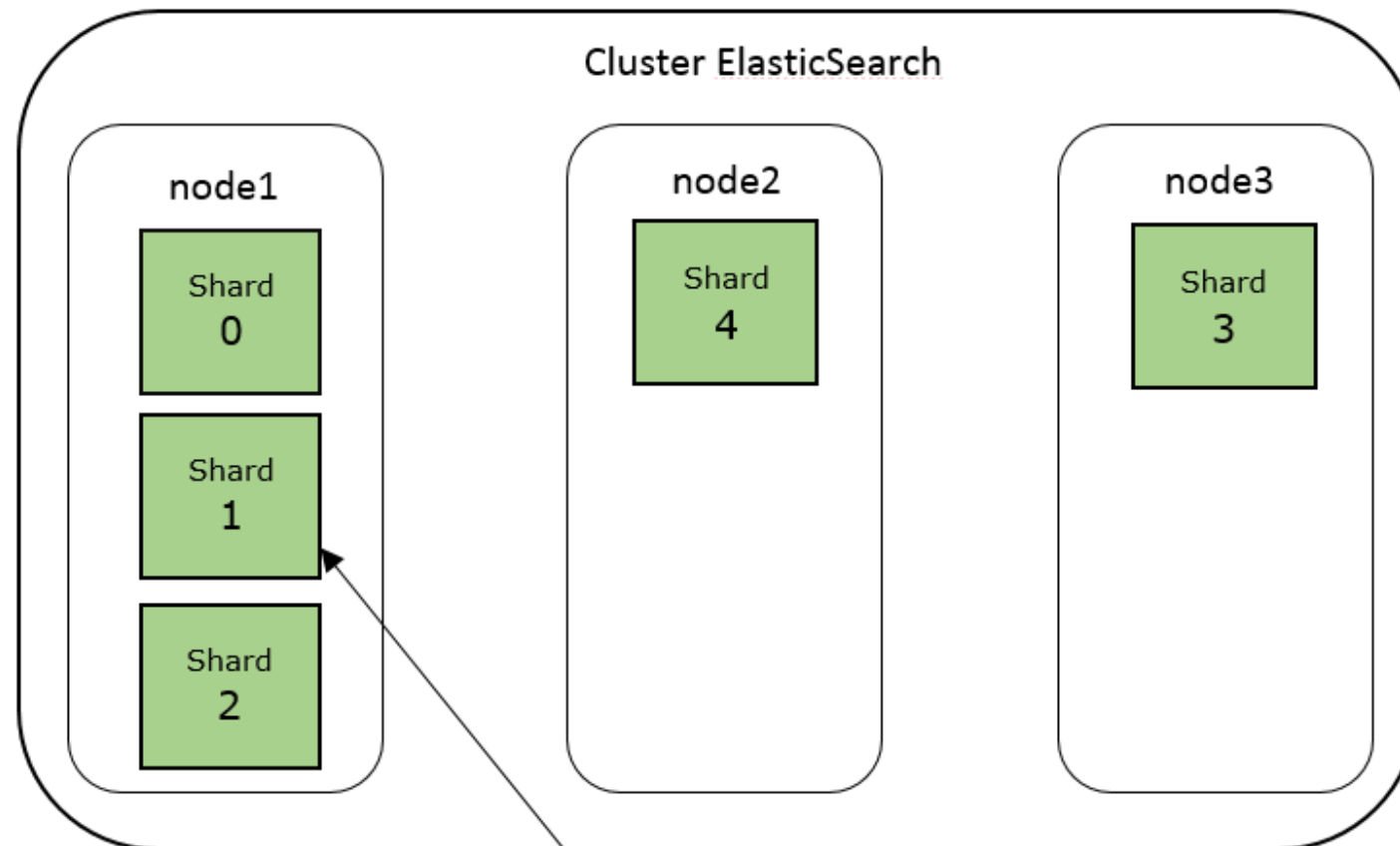
Indexation sans routage, Elasticsearch calcule une clé de hachage avec l'id du document, puis route le document vers un shard par rapport à la clé de hachage.

Indexation avec routage, on définit une valeur dans routing. Elasticsearch calcule une clé de hachage avec cette valeur.

```
curl -XPUT 'http://localhost:9200/biblio/cd/cd2?routing=opera' -d '{  
  "titre": "Le barbier de Séville",  
  "compositeur": "Gioacchino RÓSSINI"  
}'
```

```
{"ok":true,"_index":"biblio","_type":"cd","_id":"cd2","_version":1}
```



Recherche d'un document
avec routage

Application

On peut prendre la valeur de routing dans le document.
On le spécifie dans le schéma de l'index

Création d'un index

```
curl -XPUT 'http://localhost:9200/magasin'  
{ "ok":true,"acknowledged":true }
```

```
curl -XPUT 'http://localhost:9200/magasin/cd/_mapping' -d '  
{  
  "magasin":{  
    "_routing":{  
      "required":true,  
      "path":"genre"  
    }  
  }  
'
```

On crée un document avec routing=opera

```
curl -XPOST 'http://localhost:9200/magasin/cd/cd2' -d '{  
    "titre": "Le barbier de Séville",  
    "compositeur": "Gioacchino ROSSINI",  
    "genre": "opera"  
}'
```

Bibliographie :

- Site Elasticsearch
<http://www.elasticsearch.org/guide/>
<http://www.elasticsearch.cn/blog/2011/02/08/percolator.html>
<http://www.elasticsearch.org/blog/customizing-your-document-routing>
- ElasticSearch Server de Rafal Kuć et Marek Rogozinski (chapitre 1, 2, 6 et 7)
- Scaling massive elastic search clusters – Rafał Kuć
<http://www.elasticsearch.org/videos/scaling-massive-elasticsearch-clusters/>
- <http://zenika.developpez.com/tutoriels/java/installation-configuration-elasticsearch/>



elasticsearch.

Path



github

foursquare



SONY



b:
bazaarvoice

XING

salesforce
desk

IGN

mozilla

stackoverflow

KLOUT

infochimps



SCOUT 24

NETWORKED
INSIGHTS

Fog Creek
SOFTWARE

Revinate

picturesafe